

PROVISIONAL PATENT APPLICATION

Title

**METHOD AND SYSTEM FOR STATIC ANALYSIS OF
SERIALIZED PYTHON OBJECTS TO DETECT MALICIOUS
CODE WITHOUT EXECUTION**

Inventor: [INVENTOR NAME] Replace with actual inventor name
before filing

Filing Date: April 12, 2026

Application Type: Provisional Patent Application

FIELD OF THE INVENTION

This invention relates to computer security, and more particularly to static analysis methods for detecting malicious code embedded in serialized Python objects (commonly known as "pickle" files) without executing the serialized content.

BACKGROUND OF THE INVENTION

The Pickle Serialization Protocol

Python's pickle module provides object serialization capabilities, allowing arbitrary Python objects to be converted to a byte stream and later reconstructed. The protocol uses a stack-based virtual machine with opcodes that, when processed by the `pickle.load()` function, execute reconstruction operations. Key opcodes include GLOBAL (which loads a callable from a module), REDUCE (which

calls the loaded callable with arguments), BUILD (which calls `__setstate__` to set object attributes), and INST (which creates class instances).

The critical security implication arises from the `__reduce__` method. When an object defines `__reduce__`, pickle calls this method during serialization to obtain instructions for reconstruction. The return value specifies a callable and its arguments. Upon deserialization, pickle invokes this callable—meaning any callable, including `os.system`, `subprocess.Popen`, `eval`, or `exec`, can be executed with attacker-controlled arguments.

The Security Problem

Every invocation of `pickle.load()` on untrusted data constitutes arbitrary code execution. An attacker who controls a pickle file controls what code runs when that file is loaded. Unlike other serialization formats (JSON, XML, Protocol Buffers), pickle execution is not sandboxed or restricted by default.

This vulnerability has enabled numerous real-world attacks. Machine learning pipelines routinely exchange pickle files containing model weights, preprocessors, and datasets. A poisoned model file executes malicious code the moment a data scientist loads it. Supply chain attacks on package repositories (npm, PyPI) have distributed malicious packages containing pickle payloads that execute during installation or import.

Limitations of Current Approaches

Signature-based scanning compares file contents against databases of known malicious patterns. This approach fails against novel payloads—if the attacker changes any portion of their code, the signature no longer matches. Signature scanners exhibit high false-negative rates against targeted attacks.

Sandbox execution runs untrusted code in an isolated environment to observe behavior. While theoretically complete, sandbox approaches suffer from: (a) performance overhead of

virtualization, (b) anti-analysis techniques that detect sandboxes and delay malicious behavior, (c) incomplete coverage when payloads require specific conditions to trigger, and (d) resource cost of maintaining sandbox infrastructure.

AST (Abstract Syntax Tree) analysis parses Python source code to identify dangerous patterns. However, pickle files contain compiled bytecode, not source code. AST analyzers cannot process pickle files directly and miss this entire attack surface.

Static type checking and **linting tools** focus on code quality and type safety. They may warn that `pickle.load()` is dangerous in general but cannot determine whether a specific pickle file is malicious.

The Specific Threat: Supply Chain Attacks via Pickle Files

Between January 2025 and April 2026, the threat actor known as Contagious Interview (also tracked as UNC1069 and attributed to North Korean state operations) published over 1,700 malicious packages to npm and PyPI. Multiple packages contained pickle files with embedded payloads that executed on load. Traditional antivirus solutions detected zero of these packages at time of publication. The attack campaign, documented by CISA in security advisories, specifically targeted developers by embedding payloads in legitimate-appearing packages.

This threat demonstrates the inadequacy of existing detection methods for pickle-based attacks.

SUMMARY OF THE INVENTION

The present invention provides a method for analyzing Python pickle files by disassembling the pickle opcode stream and inspecting object reconstruction methods (`__reduce__`) without executing them. Rather than running code to observe behavior, the invention analyzes the structural intent encoded in pickle opcodes to determine what code WOULD execute if the pickle were loaded.

The invention detects malicious payloads by identifying dangerous callable references in GLOBAL and REDUCE opcodes, constructing a call graph of reconstruction operations, and scoring threat level based on the combination of callables, arguments, and obfuscation patterns present. This structural analysis detects novel zero-day payloads because the detection is based on what the code does (calls `os.system`), not what bytes it contains.

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENT

1. Pickle Protocol Overview

The pickle protocol encodes Python objects as a sequence of opcodes processed by a stack-based virtual machine.

Understanding this protocol is essential to the invention.

Stack-Based Execution: The pickle VM maintains a stack and a memo (dictionary for backreferences). Opcodes push values onto the stack, manipulate the stack, or store/retrieve memo entries.

Key Opcodes for Object Reconstruction:

- **GLOBAL** (opcode `c`): Pushes a callable onto the stack.
Format: `c<module>\n<name>\n`. Example: `cos\nsystem\n` loads `os.system`.
- **STACK_GLOBAL** (opcode `\x93`): Protocol 4+ variant where module and name are popped from stack rather than read from stream. Enables dynamic resolution.
- **REDUCE** (opcode `R`): Pops a callable and argument tuple from stack, calls `callable(*args)`, pushes result. This is where arbitrary code executes.
- **BUILD** (opcode `b`): Pops state and object from stack, calls `object.__setstate__(state)` or updates `object.__dict__`. Can trigger code via `__setstate__`.

- **INST** (opcode `i`): Older protocol instruction combining GLOBAL and instantiation.

The Dangerous Pattern: A GLOBAL opcode followed by REDUCE constitutes a function call. The callable loaded by GLOBAL is invoked by REDUCE with attacker-controlled arguments. If GLOBAL loads `os.system` and REDUCE passes `("rm -rf /",)`, the pickle execution runs `os.system("rm -rf /")`.

2. The Detection Method (Core of the Invention)

The invention comprises the following steps:

Step 2.1: Opcode Disassembly Without Execution

The method reads raw pickle bytes and parses them into opcode tuples WITHOUT using `pickle.load()` or any function that would execute the pickle content. The parser implements the pickle VM instruction set in read-only mode:

```
Input:  Raw bytes from pickle file
Output: List of (opcode_name, argument, byte_position) tuples
```

For each byte position, the parser:

1. Reads the opcode byte
2. Determines argument format from opcode type
3. Extracts argument (fixed-length, length-prefixed, or newline-terminated)
4. Records the triple (name, argument, position)
5. Advances to next opcode

This produces a complete disassembly without any code execution.

Step 2.2: Call Graph Construction

From the disassembly, the method builds a call graph identifying:

1. All GLOBAL opcodes and the callables they load (module.function pairs)
2. All STACK_GLOBAL opcodes (dynamic resolution, inherently suspicious)
3. All REDUCE opcodes and their relationship to preceding GLOBALs

4. The argument values passed to REDUCE operations
(extracted from preceding stack operations)

The call graph represents what functions WOULD be called with what arguments if `pickle.load()` were invoked.

Step 2.3: Threat Taxonomy Application

Each callable in the call graph is checked against a threat taxonomy organized by severity:

CRITICAL (Score 80-90): Callables that provide arbitrary code or command execution: - `os.system`, `os.popen`, `os.exec*`, `os.spawn*`
- `subprocess.call`, `subprocess.run`, `subprocess.Popen` -
`builtins.eval`, `builtins.exec`, `builtins.compile`

HIGH (Score 40-55): Callables that modify system state or import code: - `builtins.open` (file system access) - `shutil.rmtree`,
`os.remove` (destructive operations) - `importlib.import_module`,
`__import__` (code loading)

MEDIUM (Score 30-40): Callables that establish external communication: - `socket.socket` (network connections) -
`urllib.request.urlopen` (HTTP requests) -
`http.client.HTTPConnection`

Step 2.4: Obfuscation Pattern Detection

Attackers use obfuscation to evade simple scanning. The method detects:

- **Nested REDUCE operations:** Multiple REDUCE opcodes in sequence indicate callable chaining used to hide the ultimate function. Score: +40 for nested pattern.
- **Base64/encoding chains:** GLOBAL loading `base64.b64decode` or `codecs.decode` indicates encoded payloads. Score: +35.
- **getattr chains:** GLOBAL loading `builtins.getattr` enables dynamic attribute resolution to hide the target callable. Score: +25.

- **Lambda functions:** Function objects in reduce indicate runtime-constructed callables. Score: +30.
- **Unknown callables:** Any callable not in the known taxonomy requires manual review. Score: +20.

Step 2.5: Threat Score Computation

The method computes a cumulative threat score:

```
total_score = sum(score for each detected pattern)
```

Each pattern contributes its assigned score. Scores accumulate, meaning multiple medium-risk patterns can trigger a high-risk verdict.

Step 2.6: Verdict Determination

Based on the cumulative score, the method returns a verdict:

Score Range	Verdict	Recommendation
0	CLEAN	Safe to load
1-40	SUSPICIOUS	Review before loading
41-79	DANGEROUS	Do not load
80+	CRITICAL	Confirmed malicious

3. Novel Aspects Versus Prior Art

Versus Signature Scanners: Signature scanners compare byte sequences against known malicious patterns. The present invention performs structural analysis independent of specific byte content. A novel payload with never-before-seen bytes is still detected because the structure (GLOBAL loading os.system + REDUCE) reveals malicious intent. This is analogous to detecting a weapon by its function (fires projectiles) rather than its serial number.

Versus Sandbox Execution: Sandbox approaches execute code and observe behavior. The present invention NEVER executes code. This provides: (a) zero risk of escape or damage, (b) millisecond analysis time versus seconds/minutes for sandbox execution, (c) immunity to anti-analysis techniques that detect sandbox environments, and (d) no resource cost for virtualization infrastructure.

Versus AST Analysis: AST analyzers parse source code syntax trees. The present invention analyzes compiled pickle bytecode—a fundamentally different representation. There is no source code to parse; the analysis operates on the serialized opcode stream directly.

Novel Combination: The combination of (a) opcode disassembly without execution, (b) call graph construction from GLOBAL/REDUCE pairs, (c) threat taxonomy scoring, and (d) obfuscation pattern detection is not found in prior art. No existing tool performs static analysis of pickle files to detect malicious **reduce** methods without executing the pickle.

4. Implementation

The preferred embodiment implements the method in Python (approximately 400 lines) with zero external dependencies. The implementation comprises:

PickleOpParser class: Parses raw bytes into opcode tuples. Implements the pickle VM instruction set in read-only mode.

ThreatAnalyzer class: Builds call graph from parsed opcodes. Applies threat taxonomy. Detects obfuscation patterns. Computes cumulative score.

PklInspector class: Public API combining parser and analyzer. Provides `scan(filepath)` and `scan_bytes(data)` methods returning structured results.

CLI interface: Command-line tool accepting file paths and outputting formatted or JSON results.

The implementation is released as open source at github.com/johnnyg/pkl-inspector under MIT license.

5. Use Cases

5.1 ML Model Safety Verification: Before loading any pickle file containing model weights, preprocessors, or datasets, invoke pkl-inspector. Refuse to load files with DANGEROUS or CRITICAL verdicts.

5.2 CI/CD Pipeline Integration: Add pkl-inspector scan step to continuous integration pipelines. Fail builds containing malicious pickle files. Detect supply chain attacks before deployment.

5.3 Package Repository Scanning: Package repositories (PyPI, Anaconda) can scan uploaded packages for malicious pickle files before publication, preventing supply chain attacks at the source.

5.4 Incident Response Forensics: When investigating a security incident, analyze pickle files from compromised systems to understand attack payloads without risk of further infection.

5.5 Supply Chain Security Platforms: Integrate pkl-inspector into security platforms that monitor developer environments, intercepting malicious packages before execution.

CLAIMS (INFORMAL — PROVISIONAL)

Note: Formal claims are not required for provisional applications. The following informal claims describe the scope of the invention:

Claim 1: A method for detecting malicious code in Python pickle files comprising: parsing pickle file bytes to extract opcodes without execution; identifying GLOBAL opcodes that load callable references; identifying REDUCE opcodes that invoke callables; mapping each callable reference to a threat taxonomy; computing a cumulative threat score from all detected patterns; returning a verdict and detailed findings without executing any code from the pickle file.

Claim 2: The method of Claim 1 wherein the threat taxonomy classifies callables into severity levels including: CRITICAL for system execution callables (os.system, subprocess.*, eval, exec); HIGH for file system and import callables; MEDIUM for network callables; with higher severity callables contributing higher scores to the cumulative threat score.

Claim 3: The method of Claim 1 additionally comprising detection of obfuscation patterns including: multiple nested REDUCE opcodes indicating callable chaining; base64 or encoding function references indicating payload obfuscation; getattr function references indicating dynamic attribute resolution; each obfuscation pattern contributing to the cumulative threat score.

Claim 4: The method of Claim 1 wherein the parsing step extracts opcode arguments including: module and function names from GLOBAL opcodes; string and byte arguments that may contain command payloads; and these extracted values are included in the detailed findings to explain what the malicious code would execute.

Claim 5: A system implementing the method of Claims 1-4 as part of a supply chain security platform that automatically scans packages before installation, comprising: an intercept layer that captures package installation commands; a scanner component implementing the pickle analysis method; a blocking mechanism that prevents installation of packages containing malicious pickle files.

Claim 6: The system of Claim 5 wherein scan results are aggregated across multiple installations to build a crowd-sourced threat database, enabling detection of novel malicious packages based on patterns observed across the user population.

Claim 7: A computer-readable medium containing instructions that, when executed by a processor, cause the processor to perform the method of Claim 1.

ABSTRACT

A method and system for static analysis of Python pickle files to detect malicious code without execution. The method disassembles pickle bytecode into opcodes, constructs a call graph of object reconstruction methods (`__reduce__`), and applies a threat taxonomy to score dangerous patterns including system execution callables, file system operations, network operations, and obfuscation techniques. Unlike signature-based scanning which detects only known payloads, or sandbox execution which requires running potentially malicious code, this method detects novel zero-day payloads by analyzing the structural intent of the serialized code. The method processes pickle files in milliseconds with zero execution risk. Implemented as pkl-inspector, an open source Python library with zero dependencies, and integrated into the stillrunning security platform for supply chain attack prevention.

FIGURES

[Figures would be included in a formal filing. For provisional purposes, the following figure descriptions are provided:]

Figure 1: System architecture diagram showing: (a) pickle file input, (b) opcode parser, (c) call graph builder, (d) threat taxonomy matcher, (e) score calculator, (f) verdict output.

Figure 2: Flowchart of the detection method from file input to verdict output.

Figure 3: Example pickle opcode sequence showing GLOBAL loading `os.system` and REDUCE invoking it with command arguments.

Figure 4: Threat taxonomy hierarchy showing CRITICAL, HIGH, MEDIUM, and obfuscation pattern categories with associated callables.

FILING INSTRUCTIONS

To file this provisional patent application:

1. **Go to:** <https://www.uspto.gov/patents/apply>
2. **Use:** EFS-Web or Patent Center (recommended)
3. **Select:** Provisional Application for Patent
4. **Entity Status:**
5. Micro entity: < 4 prior patents, income < \$206,109 (2024 threshold)
6. Small entity: < 500 employees
7. Large entity: all others
8. **Filing Fee** (2024):
9. Micro entity: \$320
10. Small entity: \$640
11. Large entity: \$1,280
12. **Upload:** This document as the specification (PDF format)
13. **Cover Sheet:** USPTO provides form; complete with inventor name, title, correspondence address
14. **Priority Date:** Established on filing date; you have 12 months to file non-provisional

After Filing: - Receive application number and filing receipt - Product may be labeled "Patent Pending" - Begin searching for patent attorney for non-provisional - Estimated non-provisional cost: \$5,000-15,000 including USPTO fees

Recommended Timeline: - Months 1-6: Market validation, customer acquisition - Months 6-9: Engage patent attorney, draft non-provisional - Months 9-12: File non-provisional before provisional expires

INVENTOR DECLARATION

I, the undersigned inventor, declare that:

1. I believe I am the original inventor of the subject matter claimed

2. I have reviewed and understand the contents of this application
3. I acknowledge my duty to disclose material information to the USPTO

Signature: ____

Printed Name: [INVENTOR NAME]

Date: ____

This provisional patent application establishes priority date. File at uspto.gov within 12 months of disclosure.